

Retail Banking Data Model — Work Plan

1. The story — how this hangs together

The model is **one bank, one customer base, five domains reading and writing against a shared spine**.

Every domain ultimately traces back to two tables:

- **dim_customer** — who the customer is (identity, KYC, risk segment).
- **account_master** — the account itself, tagged by a class flag (**deposit** / **card** / **loan**) that determines which domain-specific subtype table it fans out to.

A single synthetic customer walks through all five domains naturally: they hold a **savings account and a term deposit** (Deposits), carry a **debit and credit card** (Cards), take an **auto loan and later a home loan** (Lending), make **UPI/NEFT payments and pay their card bill** (Payments), and have a **CIBIL score** that gated their loan approval, feeding into **internal risk rating and ECL provisioning** (Finance & Risk).

End-to-end flow, domain by domain:

1. **Customer & Account (spine)** — a customer is onboarded, KYC'd, and assigned a risk segment; one or more accounts are opened against them, each tagged with a class (deposit/card/loan) and linked to a branch and product.
2. **Retail Deposits** — deposit-class accounts carry balances and generate ledger transactions (deposits, withdrawals, interest postings) across channels (branch, ATM, net-banking, UPI).
3. **Cards** — card-class accounts are linked to physical/virtual cards; every swipe generates a card transaction against a merchant, which in turn can generate interchange fees, reward points, or a dispute.
4. **Lending Lifecycle** — a customer applies for a loan (an application, not yet an account); if approved it becomes a loan account with a repayment schedule; missed payments produce monthly delinquency snapshots, which can trigger a collections case, and in the worst case a write-off.
5. **Payments** — money moves out of (or into) any account via a payment on a specific rail (UPI/NEFT/RTGS/IMPS/SWIFT/bill pay), with its own status lifecycle (initiated → settled/failed/returned).
6. **Finance & Risk** — sits alongside all of it: bureau scores and internal risk ratings score the customer and gate loan decisions; delinquency data feeds expected-credit-loss (ECL) provisioning; fraud/AML rules watch transactions; and every domain's financial movement ultimately posts to the **general ledger**, the single source of financial truth used to prove the books reconcile.

2. Design decisions

Decision	Choice	Why
Model style	Medallion architecture — bronze (raw) → silver (conformed) → gold (star schema)	Standard, lets a data-quality layer operate cleanly at the bronze→silver boundary

Decision	Choice	Why
Fabric surfaces	Lakehouse (bronze + silver, Spark/Delta) + Warehouse (gold, T-SQL)	Lakehouse for engineering/transformation, Warehouse for BI serving
Orchestration	Data Factory pipelines	Native Fabric orchestration
Volume	~ 2.3M rows total , concentrated in the transaction fact tables (deposits, card, payments, loan) rather than spread evenly	Realistic <i>shape</i> (skewed, high-volume facts vs. small dimensions) matters more than an even row count — this is what a real bank's data actually looks like
Data generation	Custom Python scripts (Faker + a custom Indian-locale provider), seeded/reproducible, generated in strict dependency order so the baseline dataset is clean by construction	Faults will be introduced deliberately later — the baseline must be provably clean first

Noted but intentionally not included in this document: a `dq_quarantine` / `dq_correction_log` table pair to support future data-quality/self-healing work (audit trail for any row that gets flagged or auto-corrected). This is a good addition, but belongs to the next phase of the project, not this base model — flagged here so it isn't forgotten.

3. Shared spine (conformed master / dimensions)

These are referenced by every domain.

Table	Grain	Key columns
<code>dim_customer</code>	1 row / customer	customer_id PK , customer_type (indiv/SME), name, PAN , aadhaar_token, dob/incorp_date, kyc_status, kyc_level, risk_category, segment (retail/HNI/SME), occupation, income_band, customer_since, rm_employee_id FK , status
<code>dim_branch</code>	1 / branch	branch_id PK , IFSC , MICR, name, city, state, region, branch_type
<code>dim_employee</code>	1 / employee	employee_id PK , name, role (RM/loan_officer/collections_agent), branch_id FK
<code>dim_product</code>	1 / product	product_id PK , domain, product_type, name, currency, base_rate, terms_json
<code>dim_date</code>	1 / calendar day	date_key PK , date, month, quarter, fiscal_year, is_holiday
<code>dim_currency</code>	1 / currency	currency_code PK , name, symbol

Table	Grain	Key columns
<code>account_master</code>	1 / account (the spine)	<code>account_id</code> PK , <code>customer_id</code> FK (primary holder), <code>account_class</code> (deposit/card/loan), <code>product_id</code> FK , <code>branch_id</code> FK , <code>currency</code> FK , <code>open_date</code> , <code>status</code> , <code>close_date</code>
<code>bridge_account_holder</code>	1 / holder- account link	<code>account_id</code> FK , <code>customer_id</code> FK , <code>relationship</code> (primary/joint/nominee/authorised) — composite PK

`account_class` is the discriminator that fans out to the deposit / card / loan subtype tables below.

4. Domain tables — columns, grain, and relationships

4.1 Retail Deposits

Table	Grain	Columns
<code>deposit_account</code>	1 / deposit account	<code>account_id</code> (PK/FK), <code>deposit_type</code> (checking/savings/term/overdraft), <code>current_balance</code> , <code>available_balance</code> , <code>interest_rate</code> , <code>min_balance</code>
<code>term_deposit</code>	1 / term deposit	<code>account_id</code> (FK), <code>principal</code> , <code>rate</code> , <code>tenure_months</code> , <code>start_date</code> , <code>maturity_date</code> , <code>payout_freq</code> , <code>maturity_instruction</code> , <code>maturity_amount</code>
<code>overdraft_facility</code>	1 / OD facility	<code>account_id</code> (FK), <code>sanctioned_limit</code> , <code>drawn_amount</code> , <code>od_rate</code> , <code>review_date</code>
<code>fact_deposit_txn</code>	1 / ledger transaction	<code>txn_id</code> (PK), <code>account_id</code> (FK), <code>txn_datetime</code> , <code>value_date</code> , <code>amount</code> , <code>dr_cr</code> , <code>running_balance</code> , <code>channel</code> (branch/atm/netbanking/mobile/upi), <code>txn_code</code> , <code>narration</code> , <code>counterparty</code>
<code>standing_instruction</code>	1 / SI	<code>si_id</code> (PK), <code>account_id</code> (FK), <code>beneficiary</code> , <code>amount</code> , <code>frequency</code> , <code>next_run</code> , <code>status</code>
<code>interest_posting</code>	1 / accrual period	<code>posting_id</code> (PK), <code>account_id</code> (FK), <code>period</code> , <code>accrued</code> , <code>posted</code> , <code>post_date</code>

Relationships: `account_master` (`class=deposit`) → `deposit_account` (1:1) → `term_deposit` / `overdraft_facility` (1:1, conditional on type); `deposit_account` → `fact_deposit_txn`, `standing_instruction`, `interest_posting` (all 1:many).

4.2 Cards

Table	Grain	Columns
<code>card_master</code>	1 / card	<code>card_id</code> (PK), <code>account_id</code> (FK), <code>customer_id</code> (FK), <code>card_type</code> (credit/debit/prepaid), <code>network</code>

Table	Grain	Columns
		(visa/mastercard/rupay), card_token, issue_date, expiry_date, status, credit_limit, available_credit
card_account	1 / credit card	card_id (FK), statement_balance, min_due, payment_due_date, total_limit, cash_limit
dim_merchant	1 / merchant	merchant_id (PK), name, mcc, category, acquirer_id (FK), city, country
dim_acquirer	1 / acquirer	acquirer_id (PK), name
fact_card_txn	1 / card transaction	card_txn_id (PK), card_id (FK), merchant_id (FK), auth_datetime, settle_datetime, amount, currency, txn_type (purchase/atm/refund/reversal), entry_mode (pos/ecom/contactless/atm), mcc, status, auth_code
interchange_fee	1 / txn	fee_id (PK), card_txn_id (FK), interchange_amount, scheme_fee, net_merchant_amount
rewards_ledger	1 / reward event	reward_id (PK), card_id (FK), card_txn_id (FK), points_earned, points_redeemed, balance, event_date
card_statement	1 / billing cycle	statement_id (PK), card_id (FK), cycle_start, cycle_end, statement_date, opening_bal, purchases, payments, closing_bal, min_due, due_date
dispute_chargeback	1 / dispute	dispute_id (PK), card_txn_id (FK), raised_date, reason_code, dispute_amount, status (raised/represented/resolved/written_off), resolution_date, liability

Relationships: account_master (class=card) → card_master (1:1); card_master → card_account (1:1, credit only), fact_card_txn (1:many), card_statement (1:many); dim_merchant → fact_card_txn (1:many); fact_card_txn → interchange_fee (1:1), rewards_ledger (1:many), dispute_chargeback (0:1).

4.3 Lending Lifecycle

Table	Grain	Columns
loan_application	1 / application	application_id (PK), customer_id (FK), product_id (FK), requested_amount, tenure, purpose, application_date, channel, dti_ratio, cibil_at_application, decision (approved/rejected/pending), decision_date, approved_amount, officer_id (FK)
loan_account	1 / loan	loan_id (PK), application_id (FK), account_id (FK), customer_id (FK), loan_type (personal/auto/mortgage/sme), principal, rate, tenure_months, emi_amount, disbursal_date,

Table	Grain	Columns
		first_emi_date, maturity_date, outstanding_principal, status (active/closed/written_off), branch_id (FK)
repayment_schedule	1 / installment	schedule_id (PK), loan_id (FK), installment_no, due_date, emi_amount, principal_comp, interest_comp, opening_bal, closing_bal, status (due/paid/overdue)
fact_loan_txn	1 / loan transaction	loan_txn_id (PK), loan_id (FK), txn_date, amount, txn_type (disbursal/emi/prepayment/charge/waiver), principal_paid, interest_paid, dr_cr
collateral	1 / collateral item	collateral_id (PK), loan_id (FK), type (property/vehicle/other), valuation, valuation_date, ltv
delinquency_snapshot	1 / loan / month	snapshot_id (PK), loan_id (FK), snapshot_date, dpd, dpd_bucket (0/1-30/31-60/61-90/90+), overdue_amount, asset_class (standard/SMA0/SMA1/SMA2/substandard/doubtful/loss)
collection_case	1 / case	case_id (PK), loan_id (FK), open_date, status, assigned_agent_id (FK), strategy
collection_action	1 / action	action_id (PK), case_id (FK), action_date, action_type (call/sms/field_visit/notice), outcome, ptp_date, ptp_amount
write_off	1 / write-off	writeoff_id (PK), loan_id (FK), writeoff_date, writeoff_amount, recovery_amount, reason

Relationships: dim_customer → loan_application (1:many) → loan_account (1:1, on approval); account_master (class=loan) → loan_account (1:1); loan_account → repayment_schedule, fact_loan_txn, collateral, delinquency_snapshot (all 1:many); delinquency_snapshot → collection_case (may trigger) → collection_action (1:many); loan_account → write_off (0:1).

4.4 Payments

Table	Grain	Columns
dim_beneficiary	1 / payee	beneficiary_id (PK), customer_id (FK), name, account_no_or_vpa, ifsc, bank, type (within/neft/imps/upi/intl), added_date
fact_payment	1 / payment	payment_id (PK), from_account_id (FK), beneficiary_id (FK), rail (neft/rtgs/imps/upi/nach/bbps/swift/internal), amount, currency, initiated_dt, status (initiated/pending/settled/returned/failed), completed_dt, channel, reference_no
upi_detail	1 / UPI payment	payment_id (FK), payer_vpa, payee_vpa, upi_ref, psp, device_id

Table	Grain	Columns
swift_detail	1 / cross-border payment	payment_id (FK), sender_bic, receiver_bic, correspondent_bic, fx_rate, charge_bearer (OUR/BEN/SHA), purpose_code
bill_payment	1 / bill	payment_id (FK), biller_id (FK), biller_category, consumer_no, bill_amount
dim_biller	1 / biller	biller_id (PK), name, category (electricity/telecom/dth/gas/insurance), bbps_id
payment_status_event	1 / lifecycle event	event_id (PK), payment_id (FK), event_dt, status, reason_code
settlement	1 / settlement batch	settlement_id (PK), rail, settlement_date, batch_ref, total_amount, txn_count

Relationships: account_master → fact_payment (1:many, "pays from"); dim_beneficiary → fact_payment (1:many, "pays to"); fact_payment → upi_detail / swift_detail / bill_payment (0:1, conditional on rail); fact_payment → payment_status_event (1:many); fact_payment → settlement (many:1).

4.5 Finance & Risk

Table	Grain	Columns
credit_bureau_score	1 / bureau pull	score_id (PK), customer_id (FK), bureau (cibil/experian/equifax/crif), score (300–900), band, pull_date, enquiry_purpose
credit_enquiry	1 / enquiry	enquiry_id (PK), customer_id (FK), enquiry_date, type, lender, amount
internal_risk_rating	1 / rating	rating_id (PK), customer_id (FK), grade, pd_estimate, rating_date, model_version
provision_ecl	1 / loan / reporting date	provision_id (PK), loan_id (FK), reporting_date, ind_as_stage (1/2/3), ead, pd, lgd, ecl_amount
fraud_alert	1 / alert	alert_id (PK), customer_id (FK), entity_type, entity_id, alert_dt, rule_code, risk_score, status, disposition
aml_alert	1 / alert	aml_id (PK), customer_id (FK), alert_date, scenario, risk_score, status, sar_filed
exposure_limit	1 / customer / domain	limit_id (PK), customer_id (FK), product_domain, sanctioned_limit, utilised, available, as_of_date
general_ledger	1 / GL code	gl_code (PK), name, category

Table	Grain	Columns
<code>gl_entry</code>	1 / posting	<code>gl_entry_id</code> (PK), <code>gl_code</code> (FK), <code>source_txn_id</code> , <code>source_domain</code> , <code>amount</code> , <code>dr_cr</code> , <code>posting_date</code>

Relationships: `dim_customer` → `credit_bureau_score`, `credit_enquiry`, `internal_risk_rating`, `fraud_alert`, `aml_alert`, `exposure_limit` (all 1:many); `loan_account` → `provision_ecl` (1:many); `general_ledger` → `gl_entry` (1:many); every domain's fact table (`fact_deposit_txn`, `fact_card_txn`, `fact_loan_txn`, `fact_payment`) posts into `gl_entry` (1:many each) — this is the financial backbone.

Key cross-domain governance links (not just structural FKs):

- `credit_bureau_score` → gates the decision on `loan_application`
- `delinquency_snapshot` → drives `provision_ecl`
- every domain fact table → reconciles against `gl_entry`

5. Medallion → Fabric mapping

- **Bronze (Lakehouse, Delta, append-only)** — raw landing of each synthetic source extract, schema-on-read, with ingestion metadata (`source`, `load_ts`, `batch_id`, `source_row_hash`). No cleansing at this layer.
- **Silver (Lakehouse, Delta)** — conformed and trustworthy: typed, deduplicated, referential integrity enforced, the `dim_customer` / `account_master` spine conformed, SCD2 on slowly-changing dimensions, standardised codes.
- **Gold (Warehouse, T-SQL + semantic model)** — business-facing star schemas per domain plus a cross-domain "customer 360" mart, feeding Power BI.

6. Data generation & volume

Approach: Python + `Faker` with a custom Indian-locale provider (PAN, Aadhaar token, IFSC, MICR, +91 mobiles, UPI VPA, Indian names/cities/states). Seeded RNG, config-driven row counts, fully reproducible. Tables are generated in strict dependency order (dimensions → spine → domain subtype tables → fact tables → downstream risk/ledger tables) so the baseline dataset is referentially clean by construction — any faults are introduced deliberately later, never by accident.

Volume plan:

Table	Rows
<code>dim_customer</code>	50,000
<code>account_master</code>	~110,000
<code>card_master</code>	~55,000
<code>loan_application</code> / <code>loan_account</code>	35,000 / 18,000
<code>fact_deposit_txn</code>	500,000
<code>fact_card_txn</code>	300,000

Table	Rows
fact_payment	200,000
fact_loan_txn	120,000
repayment_schedule	~430,000
delinquency_snapshot	~120,000
credit_bureau_score	~80,000
Statements / rewards / disputes / collections / provisions / GL (combined)	~300,000
Total	~2.3M